UAV PHÂN LOẠI RÁC THẢI BẰNG YOLO TRÊN ROS 2 + GAZEBO

Hướng dẫn cài đặt và chạy

Cài hệ thống lên máy Windows mới, chạy demo mỗi ngày, xử lý lỗi thường gặp.

Phiên bản 1.0 · 06/11/2026 · Đội phát triển

MỤC LỤC

1. Tổng quan 3
2. Chuẩn bị máy 4
2.1. Yêu cầu máy 4
2.2. Phần mềm cần cài trên Windows 4
3. Cài đặt lần đầu 5
3.1. Cài WSL và Ubuntu 24.04 5
3.2. Lấy mã nguồn 5
3.3. Chạy script cài đặt 6
3.4. Khởi động lại Ubuntu 6
3.5. Kiểm tra môi trường 6
4. Tham số và biến môi trường 8
5. Chạy demo 9
5.1. Bật hệ thống 9
5.2. Các cách chạy khác 10
5.3. Kiểm tra hệ thống đã chạy đúng 10
5.4. Ghi video demo 11
6. Mỗi ngày bật máy 12
7. Lỗi thường gặp 13
8. Sao lưu, khôi phục 14
8.1. Sao lưu toàn bộ Ubuntu thành một file 14
8.2. Khôi phục sang máy khác hoặc sau khi cài lại Windows 14
8.3. Cài lại sạch 14
9. Tóm tắt lệnh 15

1. Tổng quan

Hệ thống chạy hoàn toàn trên một máy tính Windows, không cần drone hay camera thật. Phần mô phỏng và AI chạy trong **Ubuntu 24.04** cài bên trong Windows bằng **WSL 2** (Windows Subsystem for Linux). Các cửa sổ đồ họa của Ubuntu hiện thẳng trên màn hình Windows như ứng dụng bình thường.

Thành phần	Vai trò	Nằm ở đâu (trong Ubuntu)
WSL 2 + Ubuntu 24.04	Môi trường Linux chạy trong Windows	Cài bằng lệnh `wsl --install`
ROS 2 Jazzy	Truyền ảnh và lệnh giữa các phần của hệ thống	`/opt/ros/jazzy`
Gazebo Harmonic	Mô phỏng UAV bay trên bãi rác, sinh ảnh camera	Cài kèm ROS 2
Môi trường Python (PyTorch, YOLO)	Chạy mô hình AI nhận dạng 4 loại rác	`~/uav_venv`
Workspace ROS 2	Các node của dự án đã build	`~/uav_ws`
Mã nguồn	Code, model đã huấn luyện, script cài và chạy	`~/uav-phan-loai-rac-yolo-ros2`

Khi chạy demo sẽ mở 3 cửa sổ:

Cửa sổ	Nội dung
Gazebo Sim	Thế giới 3D: bãi cỏ 40×40 m, 64 vật thể rác, UAV X3 tự cất cánh và bay quét zig-zag ở độ cao 3 m
RViz	Ảnh camera gốc, ảnh kết quả YOLO và quỹ đạo bay của UAV
UAV Waste Monitor	Bảng giám sát: ảnh gốc, ảnh có khung nhận dạng, số lượng từng loại rác, FPS, độ trễ

Toàn bộ hệ thống bật bằng **một lệnh** `bash scripts/run_demo.sh` và tắt bằng **Ctrl+C** trong cửa sổ đó. Hệ thống không dùng cơ sở dữ liệu, không mở cổng mạng, không cần file `.env`.

2. Chuẩn bị máy

2.1. Yêu cầu máy

Hạng mục	Yêu cầu	Ghi chú
Hệ điều hành	Windows 11, hoặc Windows 10 bản 19044 trở lên	Mức tối thiểu để chạy ứng dụng đồ họa Linux (WSLg) theo tài liệu Microsoft
Ổ đĩa trống	Từ **25 GB** trở lên	Bản cài thử chiếm khoảng 16 GB (riêng thư viện Python 6,2 GB), cần thêm chỗ cho file tạm khi cài
Card đồ họa	Nên có NVIDIA; không có vẫn chạy được	Có NVIDIA: AI chạy trên GPU ~15-25 ms/ảnh. Không có: tự chuyển sang CPU ~60 ms/ảnh
Internet	Cần trong lúc cài	Tải vài GB (ROS 2, Gazebo, PyTorch). Chạy demo không cần mạng
Quyền	Tài khoản Windows có quyền Administrator	Để bật tính năng WSL

Máy đã dùng để chạy thử tài liệu này: Windows 11 Pro, NVIDIA RTX 4060 8 GB, WSL 2.4.11.

2.2. Phần mềm cần cài trên Windows

Phần mềm	Tải ở đâu	Kiểm tra sau khi cài
Driver card đồ họa (NVIDIA)	https://www.nvidia.com/drivers	Mở PowerShell gõ `nvidia-smi`, thấy tên card
WSL 2 + Ubuntu 24.04	Cài bằng lệnh ở mục 3.1 (không cần tải tay)	`wsl -l -v` thấy `Ubuntu-24.04` cột VERSION là `2`

Máy dùng card Intel hoặc AMD thì cài driver mới nhất của hãng, các link có trong trang hướng dẫn của Microsoft: https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps

3. Cài đặt lần đầu

3.1. Cài WSL và Ubuntu 24.04

Bắt buộc dùng Ubuntu 24.04. ROS 2 Jazzy chỉ hỗ trợ chính thức Ubuntu 24.04 (Tier 1, theo REP 2000: https://www.ros.org/reps/rep-2000.html). Lệnh `wsl --install` để trống tên sẽ cài bản Ubuntu mặc định, có thể là bản mới hơn (26.04) và **không dùng được** cho dự án này.

1. Bấm Start, gõ **PowerShell**, chuột phải **Windows PowerShell** → **Run as administrator**.
2. Chạy lệnh dưới đây. Nếu máy chưa từng có WSL, khởi động lại máy khi được yêu cầu.
3. Sau khi cài xong (hoặc sau khi khởi động lại), cửa sổ Ubuntu tự mở và hỏi **tên người dùng** và **mật khẩu** Linux. Đặt tên viết thường, không dấu, không khoảng trắng (ví dụ `khach`). **Ghi lại mật khẩu**: script cài đặt sẽ hỏi lại.

```
PowerShell (Administrator)
wsl --install -d Ubuntu-24.04
```

- Ổ C: còn ít chỗ thì đặt Ubuntu sang ổ khác bằng lệnh: `wsl --install Ubuntu-24.04 --location D:\WSL\Ubuntu-24.04 --web-download` (thay `D:` bằng ổ còn trống). Ổ chứa Ubuntu bị đầy sẽ làm Ubuntu chuyển sang chế độ chỉ đọc.
- Cài bị treo ở 0% thì thêm `--web-download` vào cuối lệnh (theo hướng dẫn của Microsoft).

Kiểm tra trong PowerShell:

```
PowerShell · kết quả đúng (STATE là Running nếu Ubuntu đang mở)
wsl -l -v

  NAME            STATE           VERSION
* Ubuntu-24.04    Stopped         2
```

Từ giờ, mở Ubuntu bằng cách bấm Start và gõ **Ubuntu 24.04**, hoặc gõ `wsl -d Ubuntu-24.04` trong PowerShell. Mọi lệnh ở các mục sau đều gõ trong **cửa sổ Ubuntu** (dấu nhắc dạng `khach@TEN-MAY:~$`), không gõ trong PowerShell.

```
Cửa sổ Ubuntu · kiểm tra phiên bản
cat /etc/os-release | head -2

PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
```

3.2. Lấy mã nguồn

```
Cửa sổ Ubuntu
cd ~
git clone https://github.com/vovanhung-dev/uav-phan-loai-rac-yolo-ros2.git
cd uav-phan-loai-rac-yolo-ros2
```

- Ubuntu 24.04 đã có sẵn `git`, không cần cài thêm.
- Đặt mã nguồn trong thư mục nhà của Ubuntu (`~`, tức `/home/<tên>/`), như lệnh trên. Không đặt trong `/mnt/c/...` (ổ C của Windows) vì đọc file chậm hơn nhiều.
- Nhận mã nguồn dạng file ZIP thì chép file vào Ubuntu rồi giải nén ở `~`, đảm bảo được thư mục `~/uav-phan-loai-rac-yolo-ros2` chứa các thư mục `ai`, `ros2_ws`, `scripts`.

3.3. Chạy script cài đặt

Cửa sổ Ubuntu

```
cd ~/uav-phan-loai-rac-yolo-ros2
bash scripts/setup_ubuntu.sh
```

Script tự làm lần lượt:

1. Cài ROS 2 Jazzy, Gazebo Harmonic, `ros_gz`, `cv_bridge`, `vision_msgs`, `rqt_image_view`, `colcon` (cần mật khẩu Linux ở mục 3.1).
2. Thêm người dùng vào nhóm `video` và `render` để Gazebo dùng được card đồ họa.
3. Tạo môi trường Python `~/uav_venv` và cài PyTorch 2.6 (CUDA 12.4), Ultralytics YOLO, OpenCV... theo `requirements.txt`.
4. Build workspace ROS 2 ở `~/uav_ws`.

- **Thời gian**: máy thử có mạng ~100 MB/s mất khoảng **7 phút**. Mạng chậm có thể mất 30-60 phút. Trong lúc chạy **không đóng cửa sổ, không tắt máy**: bị ngắt giữa chừng có thể làm hỏng thư viện Python (xem lỗi số 4, mục 7).
- Script hỏi `[sudo] password for <tên>:` thì gõ mật khẩu Linux rồi Enter (chữ không hiện khi gõ, bình thường).

Kết quả đúng là 3 dòng cuối như sau:

Cửa sổ Ubuntu · kết quả đúng

```
Summary: 1 package finished [1.05s]
Cài đặt hoàn tất. Chạy demo: bash /home/khach/uav-phan-loai-rac-yolo-ros2/scripts/run_demo.sh
```

3.4. Khởi động lại Ubuntu

Quyền dùng card đồ họa (nhóm `video`, `render`) chỉ có hiệu lực sau khi mở lại Ubuntu. Đóng cửa sổ Ubuntu, rồi trong PowerShell chạy:

PowerShell

```
wsl --shutdown
```

Mở lại **Ubuntu 24.04** từ Start.

3.5. Kiểm tra môi trường

Cửa sổ Ubuntu

```
source /opt/ros/jazzy/setup.bash
source ~/uav_venv/bin/activate
python -c "import numpy, cv2, torch, ultralytics, rclpy, cv_bridge; print('numpy',
numpy.__version__, 'cv2', cv2.__version__, 'torch', torch.__version__, 'cuda',
torch.cuda.is_available(), 'ultra', ultralytics.__version__)"
pip check
groups
```

Kết quả đúng

```
numpy 1.26.4 cv2 4.10.0 torch 2.6.0+cu124 cuda True ultra 8.4.174
No broken requirements found.
khach sudo video render
```

- `cuda True`: AI chạy trên card NVIDIA. Ra `cuda False` thì máy không có card NVIDIA hoặc driver chưa cài; hệ thống vẫn chạy bằng CPU, chậm hơn.
- Dòng `groups` phải có `video render`. Thiếu thì làm lại mục 3.4.

- Lần đầu có thể thấy cảnh báo `WARNING ⚠ user config directory '/home/khach/.config/Ultralytics' is not writable, using '/tmp/Ultralytics'`: không ảnh hưởng, bỏ qua.

4. Tham số và biến môi trường

Hệ thống không có file `.env`. Có thể đổi cách chạy bằng tham số ghi sau lệnh chạy demo, dạng `ten:=gia_tri`:

Tham số	Mặc định	Ý nghĩa
`headless`	`false`	`true`: không mở cửa sổ Gazebo, mô phỏng chạy nền, nhanh hơn
`rviz`	`true`	`false`: không mở RViz
`monitor`	`true`	`false`: không mở cửa sổ UAV Waste Monitor
`detector`	`true`	`false`: tắt node AI
`patrol`	`true`	`false`: UAV không tự bay
`patrol_mode`	`scan`	`scan`: bay quét zig-zag; `hover`: đứng yên trên tâm bãi
`altitude`	`3.0`	Độ cao bay (mét)
`confidence`	`0.35`	Ngưỡng tin cậy tối thiểu để hiện kết quả nhận dạng
`model_path`	Model kèm theo (`waste_yolo.pt`)	Đường dẫn file model YOLO `.pt` khác

Biến môi trường (chỉ cần khi muốn thay đổi, không bắt buộc):

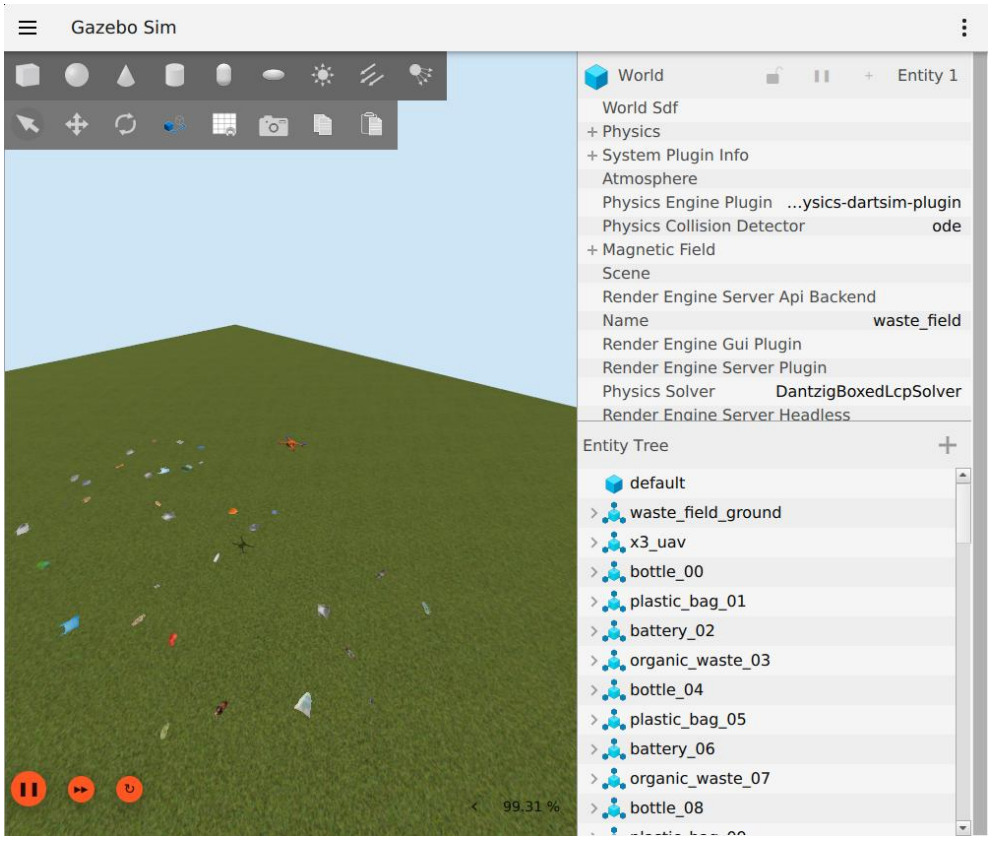
Biến	Điền gì	Để trống thì sao
`CUDA_VISIBLE_DEVICES`	Đặt rỗng (`CUDA_VISIBLE_DEVICES=""`) để ép AI chạy bằng CPU	Tự dùng GPU nếu có
`GALLIUM_DRIVER`	`d3d12` để Gazebo vẽ bằng card đồ họa qua WSL	Launch file tự đặt `d3d12` khi chạy trong WSL
`UAV_VENV`	Đường dẫn môi trường Python khác	Dùng `~/uav_venv`
`UAV_DATA_ROOT`	Thư mục dữ liệu khi huấn luyện lại AI	Dùng `~/uav_data`

5. Chạy demo

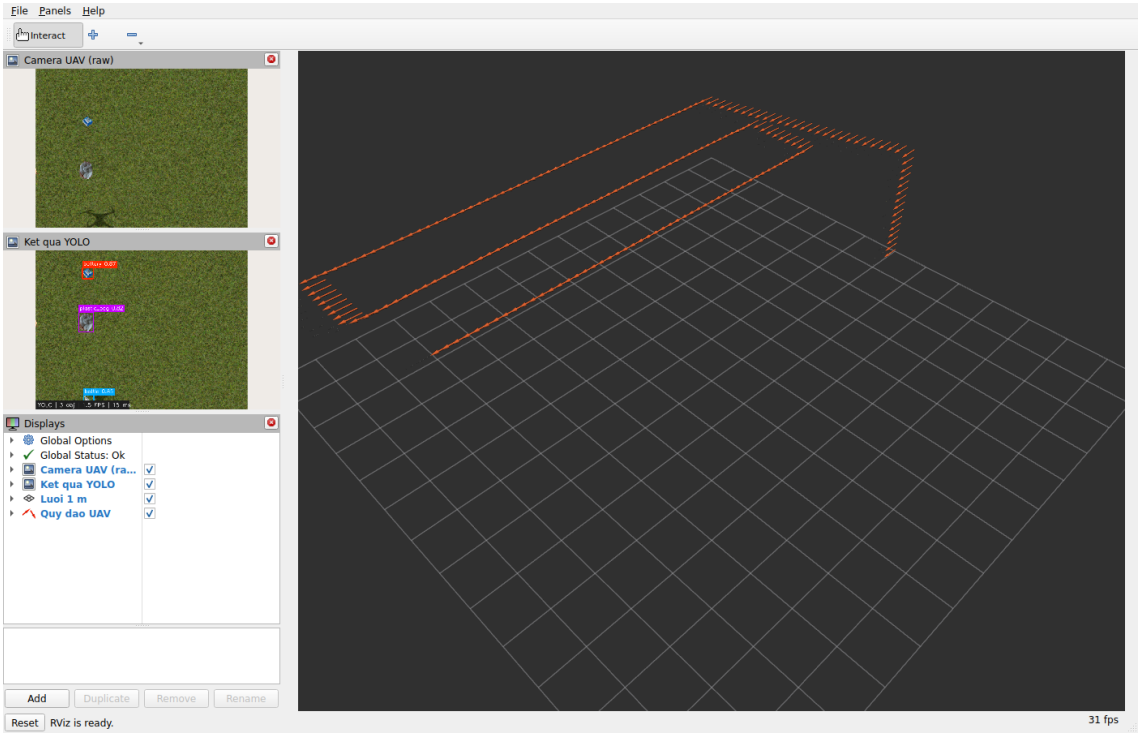
5.1. Bật hệ thống

```
Cửa sổ Ubuntu 1 · giữ mở trong lúc chạy
cd ~/uav-phan-loai-rac-yolo-ros2
bash scripts/run_demo.sh
```

- Sau khoảng 5-15 giây mở 3 cửa sổ: **Gazebo Sim**, **RViz**, **UAV Waste Monitor**.
- Cửa sổ lệnh in dòng `Đã nạp model ... trên cuda:0` (máy không có GPU: `trên cpu`), rồi `Đã cất cánh, bắt đầu di chuyển`, sau đó cứ 30 khung in một dòng thống kê, ví dụ `Khung 90: {'plastic_bag': 3, 'battery': 1, 'organic_waste': 1} | 2.9 FPS | 24 ms`.
- Các dòng cảnh báo màu vàng của Gazebo (`[Wrn] ... Ogre material scripts`, `Missing material for shape[ground]`) và `libEGL warning` là bình thường, bỏ qua.
- **Tắt**: bấm vào cửa sổ Ubuntu 1, nhấn **Ctrl+C**, chờ vài giây đến khi hiện lại dấu nhắc lệnh.



Hình 1. Cửa sổ Gazebo Sim trên máy cài thử: bãi cỏ, các vật thể rác và UAV



Hình 2. Cửa sổ RViz: ảnh camera, kết quả YOLO và quỹ đạo quét của UAV



Hình 3. Cửa sổ UAV Waste Monitor: ảnh gốc, kết quả YOLO và thống kê theo loại

Trong cửa sổ UAV Waste Monitor: phím **s** chụp ảnh màn hình giám sát, phím **q** đóng cửa sổ này.

5.2. Các cách chạy khác

```
Cửa sổ Ubuntu 1
bash scripts/run_demo.sh headless:=true monitor:=false rviz:=false
CUDA_VISIBLE_DEVICES="" bash scripts/run_demo.sh headless:=true monitor:=false rviz:=false
```

- Dòng 1: chỉ chạy mô phỏng và AI, không mở cửa sổ nào. Trên máy thử, camera đạt ~9,5 ảnh/giây (so với ~1-4 ảnh/giây khi mở đủ cửa sổ, do phần vẽ 3D của Gazebo qua WSL tốn thời gian, không phải do AI).
- Dòng 2: như dòng 1 nhưng ép AI chạy bằng CPU, để thử trước hiệu năng của máy không có card NVIDIA. Máy thử đạt ~60 ms/ảnh.

5.3. Kiểm tra hệ thống đã chạy đúng

Giữ demo đang chạy ở cửa sổ 1. Mở thêm một cửa sổ Ubuntu mới (Start → Ubuntu 24.04) và **luôn chạy 3 dòng** `source` **trước:**

Cửa sổ Ubuntu 2 · chạy mỗi khi mở cửa sổ mới

```
source /opt/ros/jazzy/setup.bash
source ~/uav_venv/bin/activate
source ~/uav_ws/install/setup.bash
```

Lệnh (cửa sổ 2)	Kết quả đúng
`ros2 node list`	Có `/gz_bridge`, `/waste_detector`, `/waste_monitor`, `/uav_patrol`, `/rviz2`, `/world_to_odom`
`ros2 topic hz /camera/image_raw`	Hiện `average rate: ...` (số ảnh/giây camera gửi lên). Nhấn Ctrl+C để dừng
`ros2 topic echo /waste_detection/summary --once`	Một dòng JSON có `"fps"`, `"latency_ms"`, `"counts"` (số rác từng loại)
`ros2 run rqt_image_view rqt_image_view /waste_detection/image_annotated`	Mở cửa sổ xem ảnh có khung nhận dạng

5.4. Ghi video demo

Cửa sổ Ubuntu 2 · khi demo đang chạy

```
ros2 run uav_waste_sim demo_recorder --ros-args -p duration:=60.0
```

Kết quả đúng: `Ghi video vào /home/khach/uav_waste_demo_<số>.mp4 trong 60 s`, hết giờ in `Đã lưu <n> khung hình vào ...`. Chép video sang thư mục Videos của Windows:

Cửa sổ Ubuntu 2

```
cp ~/uav_waste_demo_*.mp4 /mnt/c/Users/<tên-Windows>/Videos/
```

6. Mỗi ngày bật máy

Bước	Làm gì
1	Bấm Start, gõ **Ubuntu 24.04**, mở cửa sổ Ubuntu
2	`cd ~/uav-phan-loai-rac-yolo-ros2`
3	`bash scripts/run_demo.sh`
4	Dùng xong: nhấn **Ctrl+C** trong cửa sổ đó

- **Không cần** chạy lại `setup_ubuntu.sh`. Chạy lại cũng không hỏng gì (máy thử chạy lại mất 18 giây), nhưng không cần thiết.
- Tắt máy hay đóng Ubuntu đột ngột trong lúc đang chạy demo **không làm mất gì**: đã thử tắt cứng Ubuntu khi demo đang chạy, lần sau bật lại chạy bình thường.
- Ngoại lệ: tắt đột ngột **trong lúc đang chạy script cài đặt** thì xem lỗi số 4, mục 7.

7. Lỗi thường gặp

Các thông báo dưới đây được chép nguyên văn từ lần cài thử trên máy mới. khach là tên người dùng Linux của máy thử, máy khác sẽ là tên của máy đó.

#	Thông báo lỗi (nguyên văn)	Nguyên nhân	Cách xử lý
1	`ros2: command not found`	Mở cửa sổ Ubuntu mới nhưng chưa nạp môi trường ROS 2	Chạy 3 dòng `source` ở mục 5.3. Riêng lệnh chạy demo `bash scripts/run_demo.sh` đã tự nạp, không cần
2	`scripts/run_demo.sh: line 4: /home/khach/uav_ws/install/setup.bash: No such file or directory` `Package 'uav_waste_sim' not found`	Chưa chạy script cài đặt, hoặc cài đặt bị lỗi giữa chừng nên chưa build workspace	Chạy lại `bash scripts/setup_ubuntu.sh` (mục 3.3) và xem kỹ các dòng cuối
3	`An existing connection was forcibly closed by the remote host.` `Error code: Wsl/Service/0x80072746`	Dịch vụ WSL của Windows bị ngắt (gặp khi đang cài thư viện Python)	Mở lại Ubuntu, chạy lại script cài đặt. Sau đó **bắt buộc** làm bước kiểm tra ở mục 3.5; báo lỗi thì xử lý theo lỗi số 4

4	`ImportError: cannot import name 'ParamSpec' from 'typing_extensions'`
	`[ERROR] [waste_detector-4]: process has died`
	hoặc `pip check` báo hàng loạt dòng `... is not supported on this platform`
		Thư viện Python bị hỏng do cài đặt bị ngắt giữa chừng (máy thử có hơn 6.000 file bị rỗng). Chạy lại script **không** tự sửa được	Xóa môi trường Python rồi cài lại:
			`rm -rf ~/uav_venv`
			`bash scripts/setup_ubuntu.sh`
			Lệnh `rm` chỉ xóa thư viện, không xóa mã nguồn

5	`FileNotFoundError: [Errno 2] No such file or directory: '/home/khach/khong_co.pt'`	Tham số `model_path:=` trỏ tới file không tồn tại	Kiểm tra lại đường dẫn, hoặc bỏ tham số để dùng model mặc định
6	`ModuleNotFoundError: No module named 'ultralytics'` khi chạy `ros2 run uav_waste_sim waste_detector`	Chạy riêng node AI bằng `ros2 run` sẽ dùng Python hệ thống, không thấy thư viện trong `~/uav_venv`	Chạy node AI qua `bash scripts/run_demo.sh` (dùng tham số ở mục 4 để bật/tắt từng phần)

7	`Catastrophic failure`
	`Error code: Wsl/Service/E_UNEXPECTED`
	hoặc `Error code: Wsl/Service/RegisterDistro/E_UNEXPECTED`
		Dịch vụ WSL của Windows gặp lỗi khi mở hoặc đăng ký bản Ubuntu mới (gặp khi mở bản vừa cài và khi khôi phục bằng `--import`)	Chạy `wsl --shutdown` trong PowerShell rồi làm lại lệnh vừa lỗi: máy thử chạy lại `--import` thì thành công. Vẫn lỗi với bản Ubuntu khác 24.04 thì gỡ bằng `wsl --unregister <tên>` (**xóa toàn bộ dữ liệu trong bản đó**) rồi cài lại đúng `Ubuntu-24.04`

8	Gazebo, camera chạy chậm (`ros2 topic hz` chỉ ~1 ảnh/giây khi mở đủ cửa sổ)	Vẽ 3D của Gazebo qua WSL tốn tài nguyên, nhất là máy không có GPU rời	Bình thường. Muốn nhanh hơn: chạy `headless:=true` (mục 5.2), hoặc tắt RViz bằng `rviz:=false`
9	Cửa sổ lệnh in `Đã nạp model ... trên cpu`	Máy không có card NVIDIA hoặc chưa cài driver	Cài driver NVIDIA (mục 2.2), chạy `wsl --shutdown` rồi mở lại Ubuntu. Không có NVIDIA thì vẫn chạy được bằng CPU

8. Sao lưu, khôi phục

Hệ thống không có cơ sở dữ liệu. Cái cần giữ là **toàn bộ bản Ubuntu đã cài** (để khỏi cài lại) và **video/ảnh tự ghi**.

8.1. Sao lưu toàn bộ Ubuntu thành một file

```
PowerShell
wsl --shutdown
wsl --export Ubuntu-24.04 D:\SaoLuu\uav-ubuntu.tar
```

- Tạo thư mục `D:\SaoLuu` trước. File sao lưu nặng khoảng **16 GB**, máy thử mất khoảng 2 phút. Ổ chứa file sao lưu phải còn trống hơn 16 GB.
- Muốn sao lưu gọn hơn: video/ảnh tự ghi chỉ cần chép sang Windows bằng lệnh `cp` ở mục 5.4. Mã nguồn đã có trên GitHub.

8.2. Khôi phục sang máy khác hoặc sau khi cài lại Windows

```
PowerShell
wsl --import Ubuntu-24.04 D:\WSL\Ubuntu-24.04 D:\SaoLuu\uav-ubuntu.tar
wsl -d Ubuntu-24.04 -u khach
```

- Thay `khach` bằng tên người dùng Linux đã đặt ở mục 3.1. Ghi rõ `-u <tên>` để chắc chắn đăng nhập đúng người dùng, không phải `root`.
- Máy mới vẫn phải bật WSL và cài driver card đồ họa trước (mục 2.2). Không cần chạy lại script cài đặt.
- Máy thử đã khôi phục được và chạy demo bình thường ngay, không cần cài gì thêm.
- Lệnh `--import` báo `Catastrophic failure` thì xem lỗi số 7, mục 7.

8.3. Cài lại sạch

Khi môi trường hỏng nặng, gỡ bản Ubuntu rồi làm lại từ mục 3.1. Lệnh dưới **xóa toàn bộ dữ liệu trong bản Ubuntu** (gồm video đã ghi), sao lưu trước nếu cần:

```
PowerShell
wsl --unregister Ubuntu-24.04
```

9. Tóm tắt lệnh

Việc	Cửa sổ / thư mục	Lệnh
Cài WSL + Ubuntu (một lần)	PowerShell Administrator	`wsl --install -d Ubuntu-24.04`
Lấy mã nguồn (một lần)	Ubuntu, ~	`git clone https://github.com/vovanhung-dev/uav-phan-loai-rac-yolo-ros2.git`
Cài đặt (một lần)	Ubuntu, `~/uav-phan-loai-rac-yolo-ros2`	`bash scripts/setup_ubuntu.sh`
Áp dụng quyền card đồ họa (một lần)	PowerShell	`wsl --shutdown`
Chạy demo (mỗi ngày)	Ubuntu, `~/uav-phan-loai-rac-yolo-ros2`	`bash scripts/run_demo.sh`
Chạy không mở cửa sổ	Ubuntu, `~/uav-phan-loai-rac-yolo-ros2`	`bash scripts/run_demo.sh headless:=true monitor:=false rviz:=false`
Tắt demo	Cửa sổ đang chạy demo	**Ctrl+C**
Nạp môi trường ở cửa sổ mới	Ubuntu	3 dòng `source` ở mục 5.3
Ghi video 60 giây	Ubuntu (đã `source`)	`ros2 run uav_waste_sim demo_recorder --ros-args -p duration:=60.0`
Sửa thư viện Python hỏng	Ubuntu, `~/uav-phan-loai-rac-yolo-ros2`	`rm -rf ~/uav_venv` rồi `bash scripts/setup_ubuntu.sh`
Sao lưu Ubuntu	PowerShell	`wsl --export Ubuntu-24.04 D:\SaoLuu\uav-ubuntu.tar`

Tài liệu tham khảo:

- Cài WSL (Microsoft): https://learn.microsoft.com/en-us/windows/wsl/install
- Chạy ứng dụng đồ họa Linux trên WSL (Microsoft): https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps
- Nền tảng hỗ trợ của các bản ROS 2 (REP 2000): https://www.ros.org/reps/rep-2000.html
- Cài ROS 2 Jazzy trên Ubuntu: https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html
- Mã nguồn: https://github.com/vovanhung-dev/uav-phan-loai-rac-yolo-ros2